

Assignment Questions

1. Read official Spring doc (<https://docs.spring.io/spring-framework/reference/>)

Research comparison: Spring vs Java EE, Spring vs Spring Boot

2. Project Setup

Create a Maven project in Eclipse.

Add Spring Core dependencies in the pom.xml.

Ensure the project builds successfully and the Spring context loads.

3. Demonstrate IoC Container

Create a simple application with two classes, for example, Student and College.

Define beans using: XML configuration (beans.xml) & Java based configuration

Load the beans using 2 techniques of IOC container.

4. Bean Lifecycle Demonstration

Create a Teacher class with init() and destroy() methods.

Use lifecycle annotations & using interface & using xml

Configure a Java-based Spring application using
AnnotationConfigApplicationContext.

Observe the sequence of bean creation and destruction using logs.

5. Constructor Injection

Create two classes: Course and Faculty.

Inject the Course object into the Faculty constructor.

Configure the beans using: XML configuration & Java-based configuration

Print the injected data to verify successful constructor injection.

6. Setter Injection

Create two classes: Library and Book.

Inject Book into Library using setter methods.

Use both XML and Java-based configurations for practice.

Output should confirm setter injection.

7. Multiple Dependency Injection

Create a University class that depends on both Faculty and Library.

Inject both dependencies using a combination of constructor and setter injection.

Demonstrate successful wiring and access to all dependencies.

8. Bean Scope Demonstration

Create a Spring bean with prototype and singleton scope.

Demonstrate that:

Singleton beans return the same object every time.

Prototype beans return a new instance each time.

Use `@Scope("prototype")` and `@Scope("singleton")` annotations.

9. Comparison Table (Documentation Task)

Create a comparison table listing:

Constructor Injection vs Setter Injection

BeanFactory vs ApplicationContext

XML-based configuration vs Java-based configuration vs Annotation-based configuration

Include differences in usage, pros, and cons.

10. Autowiring Demonstration

Create a class Department and Employee.

Use `@Autowired` to inject Department into Employee using:

- Field injection
- Setter injection
- Constructor injection

Show how Spring resolves dependencies automatically.

11. Qualifier Annotation

Suppose you have two beans of the same type: Manager and TeamLead, both implementing a Leader interface.

Use `@Qualifier` to inject the desired bean into a Project class.

Demonstrate how ambiguity is resolved using `@Qualifier`.

12. Configuration with `@Component` and `@ComponentScan`

Create a Trainer class annotated with `@Component`.

Use `@ComponentScan` in your configuration class to detect and register it.

Print trainer details in a TrainingCenter class to verify it's auto-wired.

13. Externalize Values Using `@Value`

Create a ServerConfig class with properties like port, host, username, etc.

Use `@Value` to inject values from application.properties.

Print all property values in a method to verify external configuration.

14. SpEL Dynamic Value Injection

Create a Product class with the following fields: name (String), price (double)

discountedPrice (computed using SpEL), inStock (boolean), availabilityMessage (conditional message using SpEL)

Inject values using SpEL expressions and print the output to verify.